STUDENTS DATABASE
CRUD APPLICATION

Technology Stack: Python • FastAPI • SQLite • HTML • CSS • JavaScript

Project type	Individual backend project
Application	Students Database CRUD Application
Name	Thumpala Pavan Kalyan
Course	Future AI Agents

1. Abstract

The Students Database CRUD Application is a web-based application designed to store and manage student records. It uses Python and FastAPI to provide backend API endpoints, SQLite for persistent data storage, and HTML, CSS, and JavaScript for the browser-based interface. CRUD stands for Create, Read, Update, and Delete. The application is intended to make common student-record tasks easier to perform and demonstrate how a frontend communicates with a backend service and database.

2. Problem Statement

Managing student details manually or across separate files can make it difficult to add new records, find existing students, update incorrect information, and remove outdated records. A simple centralized application can organize these operations and reduce repetitive manual work.

3. Objectives

- Create and maintain student records in a database.
- View the list of students and retrieve an individual student record.
- Update existing student information and delete records when required.
- Validate incoming data and return clear responses for invalid or missing records.
- Provide a browser interface and interactive API documentation.
- Demonstrate integration between a frontend, REST API, and SQLite database.

4. Scope

The core scope is student-record management. A student record may include a unique student ID, name, date of birth, email and/or phone number, course, address, and enrollment date, depending on the fields implemented in the current version. An AI question-answering extension is optional and should be documented as implemented only if it has been configured and tested.

5. Technologies Used

Technology	Purpose	Role in the project
Python	Programming language	Implements application logic and database operations.
FastAPI	Backend framework	Defines API routes and validates request data.
SQLite	Database	Stores student records in a local database file.
Pydantic	Data validation	Checks request fields and data types.
HTML	Page structure	Defines dashboard layout and form fields.
CSS	Presentation	Styles the dashboard and responsive layout.
JavaScript	Frontend behavior	Sends API requests and updates the page.
Uvicorn	ASGI server	Runs the FastAPI application locally.

6. System Architecture

The application follows a simple three-part flow:

Frontend: index.html, style.css, and script.js provide the user interface.
Backend: app/main.py (and any route modules used by the current project) handles HTTP requests, validation, and responses.
Database: app/database.py manages SQLite connections and database initialization; the database file stores the records.

Typical request flow: User action in the browser → JavaScript sends an HTTP request → FastAPI processes and validates the request → SQLite reads or changes data → FastAPI returns a response → the browser updates the interface.

7. Project Folder Structure

```
AIML/
├── README.md
├── requirements.txt
└── app/
    ├── __init__.py
    ├── main.py
    ├── database.py
    ├── models.py
    ├── routers/
    └── static/
        ├── index.html
        ├── style.css
        └── script.js
```

8. Main Features

- Create: add a new student record.
- Read: view the student list and retrieve a specific student by ID.
- Update: edit information for an existing student.
- Delete: remove a student record after confirmation, if confirmation is implemented in the UI.
- API documentation: FastAPI automatically provides Swagger UI and ReDoc.

9. API Endpoints

Method	Endpoint	Purpose
GET	/	Serves the dashboard page, if configured.
POST	/students	Creates a student record.
GET	/students	Returns student records; search/filter parameters depend on implementation.
GET	/students/{student_id}	Returns one student by ID.
PUT or PATCH	/students/{student_id}	Updates a student record; use the method implemented by the app.
DELETE	/students/{student_id}	Deletes a student record.
GET	/docs	Opens interactive Swagger API documentation.
GET	/redoc	Opens ReDoc API documentation.

The exact supported methods and fields should be confirmed in Swagger UI at /docs. The table describes the intended CRUD design; it does not replace testing the current code.

10. Installation and Execution

- ✓ Open the project folder (AIML) in Visual Studio Code.
- ✓ Open Terminal → New Terminal and confirm that the prompt is in the AIML project root.
- ✓ Install dependencies using: py -m pip install -r requirements.txt
- ✓ Start the server using: py -m uvicorn app.main:app --reload
- ✓ Wait until the terminal reports that Uvicorn is running at http://127.0.0.1:8000.
- ✓ Open http://127.0.0.1:8000/ for the dashboard, http://127.0.0.1:8000/docs for Swagger UI, or http://127.0.0.1:8000/redoc for ReDoc.
- ✓ Keep the terminal open while using the application. Press Ctrl+C in the terminal to stop the server.

11. Testing Plan

Test case	Action	Expected result
Create student	Submit valid required student details.	Record is saved and a success response is shown.
Required fields	Submit without a required field.	Validation error is returned.
Duplicate ID	Create another record with an existing ID.	Duplicate is rejected if unique-ID handling is implemented.
List students	Open the student list endpoint/dashboard.	Saved records are displayed.
Find student	Request an existing student ID.	The matching record is returned.
Missing student	Request a nonexistent ID.	A not-found response is returned.
Update student	Change a student's details.	The updated values are saved and displayed.
Delete student	Delete an existing record.	The record is removed from subsequent results.
Persistence	Restart the server and list records again.	Records remain available if SQLite persistence is configured correctly.

12. Limitations and Future Enhancements

- Add user authentication and role-based access if the application will be used with real student information.
- Add pagination and more advanced search for large datasets.
- Add export to CSV or Excel and backup/restore support.
- Add automated tests for API routes and validation.
- Improve accessibility and mobile responsiveness.
- Implement and test the optional AI extension only if it is required and suitable.

13. Conclusion

The Students Database CRUD Application demonstrates the fundamental concepts of web-based data management. FastAPI provides the API layer, SQLite stores records locally, and the frontend allows users to interact with the system. The project provides a practical foundation for learning REST APIs, database operations, input validation, and frontend-backend integration.